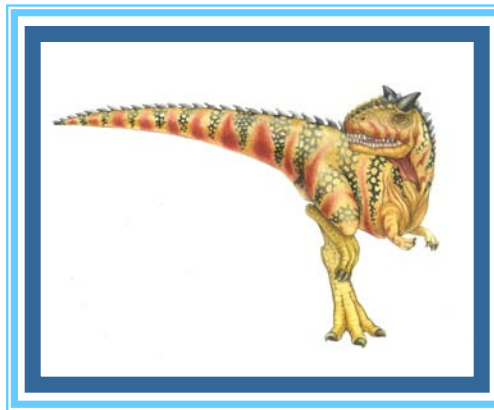


# Chapter 5: Process Synchronization

---





# Chapter 5: Process Synchronization

---

- Background
- The Critical-Section Problem
- Peterson's Solution
- Synchronization Hardware
- Mutex Locks
- Semaphores
- Classic Problems of Synchronization
- Monitors
- Synchronization Examples
- Alternative Approaches





# Objectives

---

- To present the concept of process synchronization.
- To introduce the critical-section problem, whose solutions can be used to ensure the consistency of shared data
- To present both software and hardware solutions of the critical-section problem
- To examine several classical process-synchronization problems
- To explore several tools that are used to solve process synchronization problems





# Background

---

- Processes can execute concurrently
  - May be interrupted at any time, partially completing execution
- Concurrent access to shared data may result in data inconsistency
- Maintaining data consistency requires mechanisms to ensure the orderly execution of cooperating processes
- Illustration of the problem:

Suppose that we wanted to provide a solution to the consumer-producer problem that fills **all** the buffers. We can do so by having an integer **counter** that keeps track of the number of full buffers. Initially, **counter** is set to 0. It is incremented by the producer after it produces a new buffer and is decremented by the consumer after it consumes a buffer.





# Producer

---

```
while (true) {  
    /* produce an item in next produced */  
  
    while (counter == BUFFER_SIZE) ;  
        /* do nothing */  
    buffer[in] = next_produced;  
    in = (in + 1) % BUFFER_SIZE;  
    counter++;  
}
```





# Consumer

```
while (true) {  
    while (counter == 0)  
        ; /* do nothing */  
    next_consumed = buffer[out];  
    out = (out + 1) % BUFFER_SIZE;  
    counter--;  
    /* consume the item in next consumed */  
}
```

Process synchronization is the technique of making sure that two or more processes can work together safely when they share resources. It ensures that shared data is accessed in the right order to avoid problems like race conditions.





# Race Condition

A race condition happens when two or more processes access shared data at the same time, and the final result depends on the order they run. It can cause incorrect or unpredictable results.

To prevent race conditions, processes must be synchronized using techniques like locks, semaphores, or monitors.

## Example of Race Condition:

Imagine a shared bank account with a balance of \$100. Two processes try to withdraw \$50 at the same time.

Process A reads the balance  $\rightarrow$  \$100

Process B reads the balance  $\rightarrow$  \$100

Process A subtracts \$50  $\rightarrow$  \$50

Process B subtracts \$50  $\rightarrow$  \$50

Both processes write back  $\rightarrow$  Final balance = \$50

✗ The correct final balance should be \$0, but because both processes ran at the same time without synchronization, we got the wrong result.



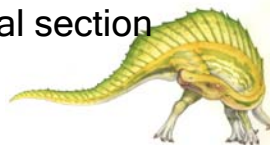


# Critical Section Problem

## Critical Section Problem (for n processes)

- Consider a system with n processes:  $\{P_0, P_1, \dots, P_{n-1}\}$ .
- Each process has a **critical section** in its code:
  - It may change shared variables, update tables, or write to files.
- **Rule:** When one process is in its critical section, no other process can enter its critical section.
- The **critical section problem** is to design a **protocol** that allows processes to safely enter and leave their critical sections.
- Structure of each process:
  1. **Entry Section** → Process asks permission to enter critical section.
  2. **Critical Section** → Process executes code that uses shared resources.
  3. **Exit Section** → Process leaves critical section.
  4. **Remainder Section** → Process does other work outside the critical section.

A critical section is a part of a program where a process accesses shared resources. The Critical Section Problem is how to design a system so that only one process executes its critical section at a time





# Critical Section

- General structure of process  $P_i$

```
do {  
    entry section  
    critical section  
    exit section  
    remainder section  
} while (true);
```





# Algorithm for Process $P_i$

---

```
do {  
    while (turn == j);  
        critical section  
    turn = j;  
        remainder section  
} while (true);
```





# Solution to Critical-Section Problem

1. **Mutual Exclusion** - If process  $P_i$  is executing in its critical section, then no other processes can be executing in their critical sections
2. **Progress** - If no process is in the critical section and some processes want to enter, The system must allow one of them to enter soon. The selection cannot be delayed forever.
3. **Bounded Waiting** - A process cannot be kept waiting forever to enter its critical section. There is a limit on how many times other processes can go before it. This ensures every process eventually gets its turn (no starvation).
  - Assume that each process executes at a nonzero speed
  - No assumption concerning **relative speed** of the  $n$  processes





# Critical-Section Handling in OS

---

Two approaches depending on if kernel is preemptive or non-preemptive

- **Preemptive** – allows preemption of process when running in kernel mode
- **Non-preemptive** – runs until exits kernel mode, blocks, or voluntarily yields CPU
  - ▶ Essentially free of race conditions in kernel mode





# Peterson's Solution

- Good algorithmic description of solving the problem
- Two process solution
- Assume that the **load** and **store** machine-language instructions are atomic; that is, cannot be interrupted
- The two processes share two variables:
  - **int** `turn`;
  - **Boolean** `flag[2]`
- The variable **turn** indicates whose turn it is to enter the critical section
- The **flag** array is used to indicate if a process is ready to enter the critical section. **flag[i] = true** implies that process **P<sub>i</sub>** is ready!





# Algorithm for Peterson's Solution

```
do {  
    flag[i] = true;  
    turn = j;  
    while (flag[j] && turn == j);  
        critical section  
    flag[i] = false;  
        remainder section  
} while (true);
```

Peterson's solution is a software-based method for two processes to achieve mutual exclusion.

1. Uses two variables: flag[i] (if process wants to enter) and turn (whose turn it is).
2. Ensures mutual exclusion, progress, and bounded waiting.





# Peterson's Solution (Cont.)

- Provable that the three CS requirement are met:

1. Mutual exclusion is preserved

$P_i$  enters CS only if:

either **flag[j] = false** or **turn = i**

2. Progress requirement is satisfied
3. Bounded-waiting requirement is met





# Synchronization Hardware

---

- Many systems provide hardware support for implementing the critical section code.
- All solutions below based on idea of **locking**
  - Protecting critical regions via locks
- Uniprocessors – could disable interrupts
  - Currently running code would execute without preemption
  - Generally too inefficient on multiprocessor systems
    - ▶ Operating systems using this not broadly scalable
- Modern machines provide special atomic hardware instructions
  - ▶ **Atomic** = non-interruptible
    - Either test memory word and set value
    - Or swap contents of two memory words





# Solution to Critical-section Problem Using Locks

---

```
do {  
    acquire lock  
        critical section  
    release lock  
        remainder section  
} while (TRUE);
```





# test\_and\_set Instruction

---

Definition:

```
boolean test_and_set (boolean *target)
{
    boolean rv = *target;
    *target = TRUE;
    return rv;
}
```

1. Executed atomically
2. Returns the original value of passed parameter
3. Set the new value of passed parameter to “TRUE”.





# Solution using test\_and\_set()

---

- Shared Boolean variable lock, initialized to FALSE
- Solution:

```
do {  
    while (test_and_set(&lock))  
        ; /* do nothing */  
        /* critical section */  
    lock = false;  
        /* remainder section */  
} while (true);
```





# compare\_and\_swap Instruction

Definition:

```
int compare_and_swap(int *value, int expected, int new_value) {  
    int temp = *value;  
  
    if (*value == expected)  
        *value = new_value;  
    return temp;  
}
```

1. Executed atomically
2. Returns the original value of passed parameter “value”
3. Set the variable “value” the value of the passed parameter “new\_value” but only if “value” == “expected”. That is, the swap takes place only under this condition.





# Solution using compare\_and\_swap

---

- Shared integer “lock” initialized to 0;
- Solution:

```
do {  
    while (compare_and_swap(&lock, 0, 1) != 0)  
        ; /* do nothing */  
    /* critical section */  
    lock = 0;  
    /* remainder section */  
} while (true);
```





# Bounded-waiting Mutual Exclusion with test\_and\_set

---

```
do {
    waiting[i] = true;
    key = true;
    while (waiting[i] && key)
        key = test_and_set(&lock);
    waiting[i] = false;
    /* critical section */
    j = (i + 1) % n;
    while ((j != i) && !waiting[j])
        j = (j + 1) % n;
    if (j == i)
        lock = false;
    else
        waiting[j] = false;
    /* remainder section */
} while (true);
```





# Mutex Locks

- Previous solutions are complicated and generally inaccessible to application programmers
- OS designers build software tools to solve critical section problem
- Simplest is mutex lock
- Protect a critical section by first **acquire()** a lock then **release()** the lock
  - Boolean variable indicating if lock is available or not
- Calls to **acquire()** and **release()** must be atomic
  - Usually implemented via hardware atomic instructions
- But this solution requires **busy waiting**
  - This lock therefore called a **spinlock**





# acquire() and release()

- A mutex is a lock used to protect shared resources so that only one thread or process can access it at a time. Mutex locks are used to lock and unlock critical sections.

## Functions:

`acquire()` → Lock the mutex.

`release()` → Unlock the mutex.

Prevents multiple threads from entering critical section at the same time.

- ```
do {  
    acquire lock  
    critical section  
    release lock  
    remainder section  
} while (true);
```





# Semaphore

- A semaphore is an integer variable used for synchronization.
- Types:
  - Binary semaphore  $\rightarrow$  0 or 1 (like a mutex).
  - Counting semaphore  $\rightarrow$  Can take non-negative values.
- Operations:
  - `wait()` / `P()`  $\rightarrow$  Decrease value (may block).
  - `signal()` / `V()`  $\rightarrow$  Increase value (may wake up waiting process).

) for

■ Definition of the **wait()** operation-request access to a resource.

```
wait(S) {  
    while (S <= 0)  
        ; // busy wait  
    S--;  
}
```

■ Definition of the **signal()** operation-to release a resource

```
signal(S) {  
    S++;  
}
```





# Semaphore Usage

- **Counting semaphore** – integer value can range over an unrestricted domain
- **Binary semaphore** – integer value can range only between 0 and 1
  - Same as a **mutex lock**
- Can solve various synchronization problems
- Consider  $P_1$  and  $P_2$  that require  $S_1$  to happen before  $S_2$   
Create a semaphore “**synch**” initialized to 0

**P1:**

```
 $S_1$ ;  
signal(synch);
```

**P2:**

```
wait(synch);  
 $S_2$ ;
```

- Can implement a counting semaphore  $S$  as a binary semaphore





# Semaphore Implementation

- Must guarantee that no two processes can execute the **wait()** and **signal()** on the same semaphore at the same time
- Thus, the implementation becomes the critical section problem where the **wait** and **signal** code are placed in the critical section
  - Could now have **busy waiting** in critical section implementation
    - ▶ But implementation code is short
    - ▶ Little busy waiting if critical section rarely occupied
- Note that applications may spend lots of time in critical sections and therefore this is not a good solution





# Semaphore Implementation with no Busy waiting

- With each semaphore there is an associated waiting queue
- Each entry in a waiting queue has two data items:
  - value (of type integer)
  - pointer to next record in the list
- Two operations:
  - **block** – place the process invoking the operation on the appropriate waiting queue
  - **wakeup** – remove one of processes in the waiting queue and place it in the ready queue
- `typedef struct{  
    int value;  
    struct process *list;  
} semaphore;`





## Implementation with no Busy waiting (Cont.)

```
wait(semaphore *S) {
    S->value--;
    if (S->value < 0) {
        add this process to S->list;
        block();
    }
}

signal(semaphore *S) {
    S->value++;
    if (S->value <= 0) {
        remove a process P from S->list;
        wakeup(P);
    }
}
```





# Deadlock and Starvation

- **Deadlock** – Two or more processes are waiting indefinitely for each other to release resources.
- Let **S** and **Q** be two semaphores initialized to 1

$P_0$   
`wait(S);`  
`wait(Q);`  
...  
`signal(S);`  
`signal(Q);`

$P_1$   
`wait(Q);`  
`wait(S);`  
...  
`signal(Q);`  
`signal(S);`

- **Starvation** – **indefinite blocking**
  - A Starvation occurs in a multitasking system when a process waits indefinitely to get the resources it needs because other processes keep getting preference. Essentially, the process is ready to execute but never gets a chance to run.
- **Priority Inversion** – Scheduling problem when lower-priority process holds a lock needed by higher-priority process
  - Solved via **priority-inheritance protocol**





# Classical Problems of Synchronization

- Classical problems used to test newly-proposed synchronization schemes
  - Bounded-Buffer Problem
  - Readers and Writers Problem
  - Dining-Philosophers Problem





# Bounded-Buffer Problem

---

- $n$  buffers, each can hold one item
- Semaphore **mutex** initialized to the value 1
- Semaphore **full** initialized to the value 0
- Semaphore **empty** initialized to the value  $n$





# Bounded Buffer Problem (Cont.)

- The structure of the producer process

```
do {  
    ...  
    /* produce an item in next_produced */  
    ...  
    wait(empty);  
    wait(mutex);  
    ...  
    /* add next produced to the buffer */  
    ...  
    signal(mutex);  
    signal(full);  
} while (true);
```





# Bounded Buffer Problem (Cont.)

- The structure of the consumer process

```
Do {  
    wait(full);  
    wait(mutex);  
    ...  
    /* remove an item from buffer to next_consumed */  
    ...  
    signal(mutex);  
    signal(empty);  
    ...  
    /* consume the item in next consumed */  
    ...  
} while (true);
```





# Readers-Writers Problem

## Readers-Writers Problem

The **Readers-Writers Problem** is a classic synchronization problem that deals with **multiple processes accessing a shared resource** such as a database. There are two types of processes: **readers** and **writers**.

- **Readers** only read the data and do not modify it.
- **Writers** can modify the data and need exclusive access.

### Rules:

1. Multiple readers can read the resource **at the same time**.
2. Only one writer can write **at a time**.
3. Readers **cannot read while a writer is writing**.

**Goal:** Allow maximum concurrency for readers while preventing conflicts and ensuring data consistency.

- Semaphore **rw\_mutex** initialized to 1
- Semaphore **mutex** initialized to 1
- Integer **read\_count** initialized to 0





# Readers-Writers Problem (Cont.)

- The structure of a writer process

```
do {  
    wait(rw_mutex);  
    ...  
    /* writing is performed */  
    ...  
    signal(rw_mutex);  
} while (true);
```





# Readers-Writers Problem (Cont.)

## ■ The structure of a reader process

```
do {  
    wait(mutex);  
    read_count++;  
    if (read_count == 1)  
        wait(rw_mutex);  
    signal(mutex);  
  
    ...  
    /* reading is performed */  
    ...  
    wait(mutex);  
    read_count--;  
    if (read_count == 0)  
        signal(rw_mutex);  
    signal(mutex);  
} while (true);
```





# Readers-Writers Problem Variations

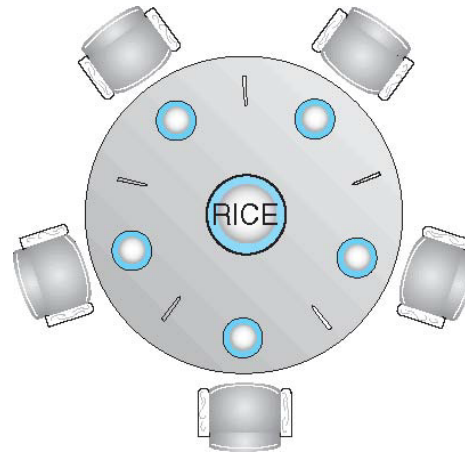
---

- **First** variation – no reader kept waiting unless writer has permission to use shared object
- **Second** variation – once writer is ready, it performs the write ASAP
- Both may have starvation leading to even more variations
- Problem is solved on some systems by kernel providing reader-writer locks





# Dining-Philosophers Problem



## Dining Philosophers Problem

The **Dining Philosophers Problem** is a classic synchronization problem that shows **how to avoid deadlock and starvation** when multiple processes share resources.

- Imagine **five philosophers sitting around a table**. Each philosopher alternates between **thinking and eating**.
- There is **one fork between each pair of philosophers**, and a philosopher needs **two forks to eat**.
- **Problem:** If all philosophers pick up the fork on their right at the same time, **no one can eat**, causing **deadlock**.
- **Goal:** Design a system so that:
  1. No deadlock occurs.
  2. No philosopher starves (everyone eventually gets to eat).
  3. Maximum concurrency is allowed (as many philosophers eat as possible at the same time).





# Dining-Philosophers Problem Algorithm

- The structure of Philosopher  $i$ :

```
do {  
    wait (chopstick[i] );  
    wait (chopStick[ (i + 1) % 5] );  
  
    // eat  
  
    signal (chopstick[i] );  
    signal (chopstick[ (i + 1) % 5] );  
  
    // think  
  
} while (TRUE);
```

- What is the problem with this algorithm?





# Dining-Philosophers Problem Algorithm (Cont.)

---

- Deadlock handling
  - Allow at most 4 philosophers to be sitting simultaneously at the table.
  - Allow a philosopher to pick up the forks only if both are available (picking must be done in a critical section).
  - Use an asymmetric solution -- an odd-numbered philosopher picks up first the left chopstick and then the right chopstick. Even-numbered philosopher picks up first the right chopstick and then the left chopstick.





# Problems with Semaphores

---

- Incorrect use of semaphore operations:
  - signal (mutex) .... wait (mutex)
  - wait (mutex) ... wait (mutex)
  - Omitting of wait (mutex) or signal (mutex) (or both)
- Deadlock and starvation are possible.





# Monitors

- A monitor is a high-level synchronization construct that combines shared variables, procedures, and automatic mutual exclusion.
- *Abstract data type*, internal variables only accessible by code within the procedure
- Only one process may be active within the monitor at a time
- But not powerful enough to model some synchronization schemes

```
monitor monitor-name
{
    // shared variable declarations
    procedure P1 (...) { ... }

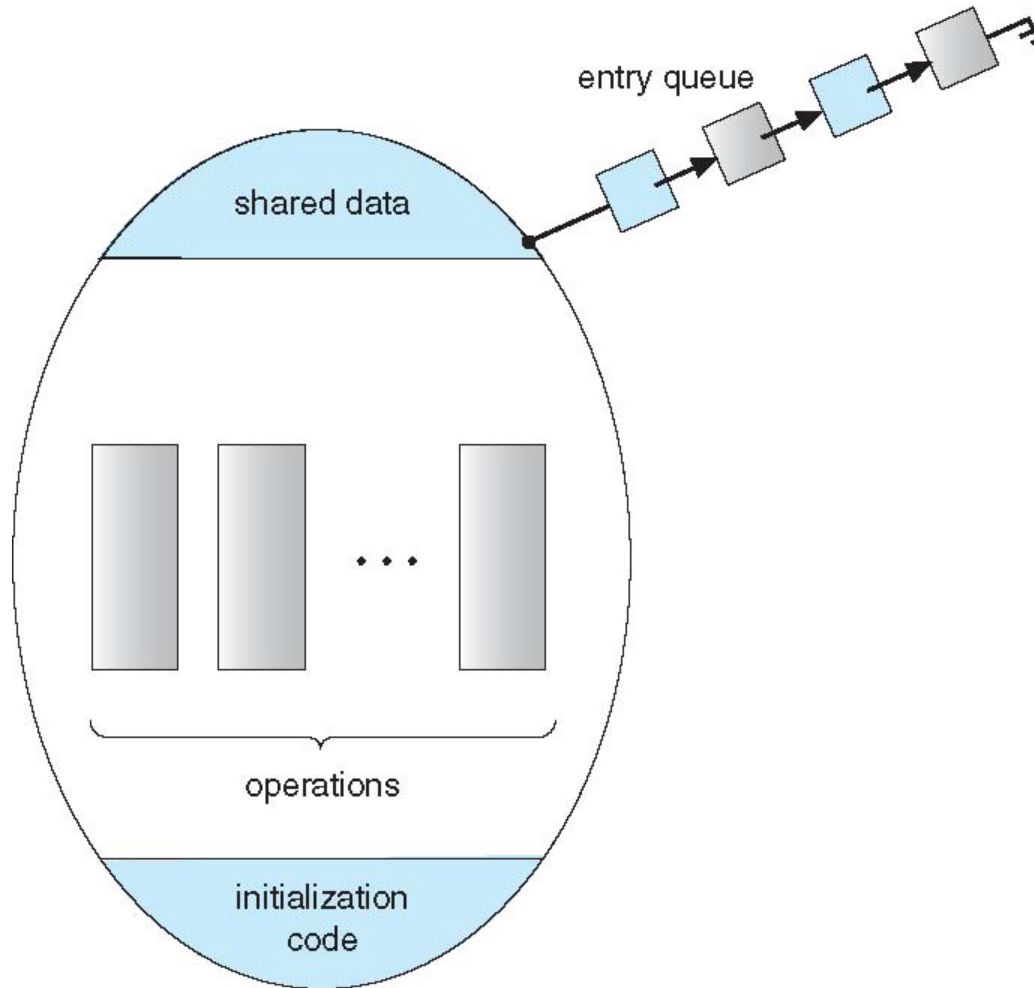
    procedure Pn (...) {.....}

    Initialization code (...) { ... }
}
}
```





# Schematic view of a Monitor





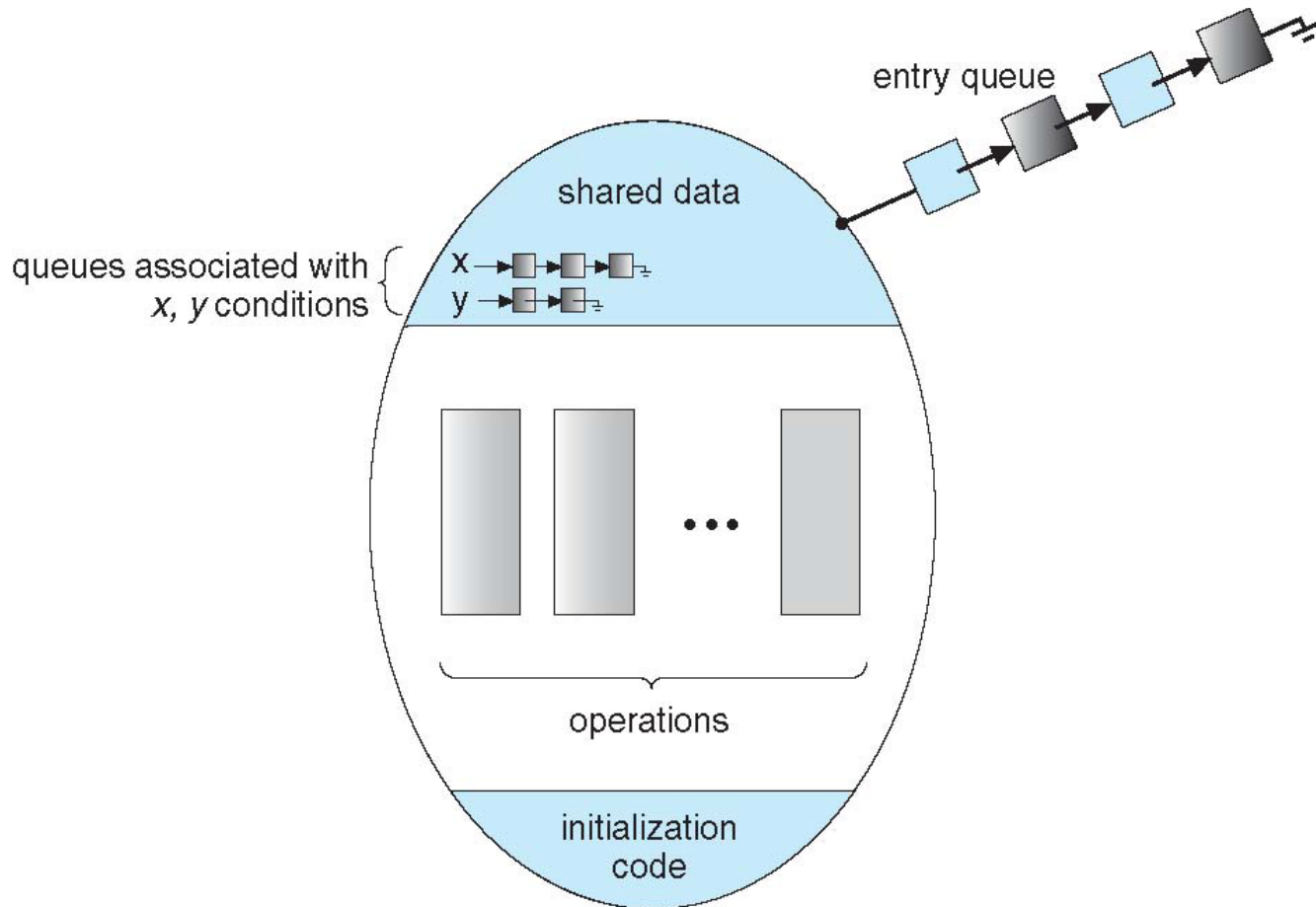
# Condition Variables

- **condition** *x*, *y*;
- Two operations are allowed on a condition variable:
  - ***x.wait()*** – a process that invokes the operation is suspended until ***x.signal()***
  - ***x.signal()*** – resumes one of processes (if any) that invoked ***x.wait()***
    - ▶ If no ***x.wait()*** on the variable, then it has no effect on the variable





# Monitor with Condition Variables





# Condition Variables Choices

- If process P invokes **x.signal()**, and process Q is suspended in **x.wait()**, what should happen next?
  - Both Q and P cannot execute in parallel. If Q is resumed, then P must wait
- Options include
  - **Signal and wait** – P waits until Q either leaves the monitor or it waits for another condition
  - **Signal and continue** – Q waits until P either leaves the monitor or it waits for another condition
  - Both have pros and cons – language implementer can decide
  - Monitors implemented in Concurrent Pascal compromise
    - ▶ P executing signal immediately leaves the monitor, Q is resumed
  - Implemented in other languages including Mesa, C#, Java





# Monitor Solution to Dining Philosophers

```
monitor DiningPhilosophers
{
    enum { THINKING; HUNGRY, EATING) state [5] ;
    condition self [5];

    void pickup (int i) {
        state[i] = HUNGRY;
        test(i);
        if (state[i] != EATING) self[i].wait;
    }

    void putdown (int i) {
        state[i] = THINKING;
        // test left and right neighbors
        test((i + 4) % 5);
        test((i + 1) % 5);
    }
}
```





# Solution to Dining Philosophers (Cont.)

```
void test (int i) {
    if ((state[(i + 4) % 5] != EATING) &&
        (state[i] == HUNGRY) &&
        (state[(i + 1) % 5] != EATING) ) {
        state[i] = EATING ;
        self[i].signal () ;
    }
}

initialization_code() {
    for (int i = 0; i < 5; i++)
        state[i] = THINKING;
}
}
```





# Solution to Dining Philosophers (Cont.)

---

- Each philosopher  $i$  invokes the operations **pickup()** and **putdown()** in the following sequence:

**DiningPhilosophers.pickup(i);**

**EAT**

**DiningPhilosophers.putdown(i);**

- No deadlock, but starvation is possible





# Monitor Implementation Using Semaphores

- Variables

```
semaphore mutex;  // (initially = 1)
semaphore next;   // (initially = 0)
int next_count = 0;
```

- Each procedure  $F$  will be replaced by

```
wait(mutex);
...
body of F;
...
if (next_count > 0)
    signal(next)
else
    signal(mutex);
```

- Mutual exclusion within a monitor is ensured





# Monitor Implementation – Condition Variables

- For each condition variable  $x$ , we have:

```
semaphore x_sem; // (initially = 0)
int x_count = 0;
```

- The operation  $x.\text{wait}$  can be implemented as:

```
x_count++;
if (next_count > 0)
    signal(next);
else
    signal(mutex);
wait(x_sem);
x_count--;
```





# Monitor Implementation (Cont.)

- The operation `x.signal` can be implemented as:

```
if (x_count > 0) {  
    next_count++;  
    signal(x_sem);  
    wait(next);  
    next_count--;  
}
```





# Resuming Processes within a Monitor

---

- If several processes queued on condition  $x$ , and  $x.\text{signal}()$  executed, which should be resumed?
- FCFS frequently not adequate
- **conditional-wait** construct of the form  $x.\text{wait}(c)$ 
  - Where  $c$  is **priority number**
  - Process with lowest number (highest priority) is scheduled next





# Single Resource allocation

- Allocate a single resource among competing processes using priority numbers that specify the maximum time a process plans to use the resource

```
R.acquire(t);  
    ...  
    access the resource;  
    ...  
  
R.release;
```

- Where R is an instance of type **ResourceAllocator**





# A Monitor to Allocate Single Resource

---

```
monitor ResourceAllocator
{
    boolean busy;
    condition x;
    void acquire(int time) {
        if (busy)
            x.wait(time);
        busy = TRUE;
    }
    void release() {
        busy = FALSE;
        x.signal();
    }
    initialization code() {
        busy = FALSE;
    }
}
```





# Synchronization Examples

---

- Solaris
- Windows
- Linux
- Pthreads





# Solaris Synchronization

- Implements a variety of locks to support multitasking, multithreading (including real-time threads), and multiprocessing
- Uses **adaptive mutexes** for efficiency when protecting data from short code segments
  - Starts as a standard semaphore spin-lock
  - If lock held, and by a thread running on another CPU, spins
  - If lock held by non-run-state thread, block and sleep waiting for signal of lock being released
- Uses **condition variables**
- Uses **readers-writers** locks when longer sections of code need access to data
- Uses **turnstiles** to order the list of threads waiting to acquire either an adaptive mutex or reader-writer lock
  - Turnstiles are per-lock-holding-thread, not per-object
- Priority-inheritance per-turnstile gives the running thread the highest of the priorities of the threads in its turnstile





# Windows Synchronization

---

- Uses interrupt masks to protect access to global resources on uniprocessor systems
- Uses **spinlocks** on multiprocessor systems
  - Spinning-thread will never be preempted
- Also provides **dispatcher objects** user-land which may act mutexes, semaphores, events, and timers
  - **Events**
    - ▶ An event acts much like a condition variable
  - Timers notify one or more thread when time expired
  - Dispatcher objects either **signaled-state** (object available) or **non-signaled state** (thread will block)





# Linux Synchronization

---

- Linux:
  - Prior to kernel Version 2.6, disables interrupts to implement short critical sections
  - Version 2.6 and later, fully preemptive
- Linux provides:
  - Semaphores
  - atomic integers
  - spinlocks
  - reader-writer versions of both
- On single-cpu system, spinlocks replaced by enabling and disabling kernel preemption





# Pthreads Synchronization

---

- Pthreads API is OS-independent
- It provides:
  - mutex locks
  - condition variable
- Non-portable extensions include:
  - read-write locks
  - spinlocks





# Alternative Approaches

---

- Transactional Memory
- OpenMP
- Functional Programming Languages





# Transactional Memory

- A **memory transaction** is a sequence of read-write operations to memory that are performed atomically.

```
void update()  
{  
    /* read/write memory */  
}
```





# OpenMP

- OpenMP is a set of compiler directives and API that support parallel programming.

```
void update(int value)
{
    #pragma omp critical
    {
        count += value
    }
}
```

The code contained within the **#pragma omp critical** directive is treated as a critical section and performed atomically.





# Functional Programming Languages

---

- Functional programming languages offer a different paradigm than procedural languages in that they do not maintain state.
- Variables are treated as immutable and cannot change state once they have been assigned a value.
- There is increasing interest in functional languages such as Erlang and Scala for their approach in handling data races.



# End of Chapter 5

---

